Reinforcement Learning-Based Joint Task Offloading and Migration Schemes Optimization in Mobility-Aware MEC Network

Dongyu Wang*, Xinqiao Tian, Haoran Cui, Zhaolin Liu

Key Laboratory of Universal Wireless Communications, Ministry of Education, Beijing University of Posts and Telecommunications, Beijing 100876, China

* The corresponding author, email: dy_wang@bupt.edu.cn

Abstract: Intelligent edge computing carries out edge devices of the Internet of things (IoT) for data collection, calculation and intelligent analysis, so as to proceed data analysis nearby and make feedback timely. Because of the mobility of mobile equipments (MEs), if MEs move among the reach of the small cell networks (SCNs), the offloaded tasks cannot be returned to MEs successfully. As a result, migration incurs additional costs. In this paper, joint task offloading and migration schemes in mobility-aware Mobile Edge Computing (MEC) network based on Reinforcement Learning (RL) are proposed to obtain the maximum system revenue. Firstly, the joint optimization problems of maximizing the total revenue of MEs are put forward, in view of the mobility-aware MEs. Secondly, considering time-varying computation tasks and resource conditions, the mixed integer non-linear programming (MINLP) problem is described as a Markov Decision Process (MDP). Then we propose a novel reinforcement learning-based optimization framework to work out the problem, instead traditional methods. Finally, it is shown that the proposed schemes can obviously raise the total revenue of MEs by giving simulation results.

Keywords: MEC; computation offloading; mobility-aware; migration scheme; Markov decision process; reinforcement learning

I. INTRODUCTION

In recent years, intelligent terminals become more and more common in the mobile network and various new applications such as AR/VR, image recognition and biometric identification emerge in an endless stream. These applications are usually resource-intensive, which need to consume a large amount of computation resource while running, so they have high requirements on Quality of Service (QoS) [1] [2]. Although the performance of intelligent terminal processors is constantly enhanced, it's still difficult to meet the demand of processing high-performance applications in a short time, which seriously affects the Quality of Experience (QoE) of devices [3]. Therefore, the urgent problem is how to expand the intelligent terminal resources to meet the high-performance task execution demand [4] [5].

Cloud computing provides an economical and efficient solution for mass data storage and process. Mobile cloud computing (MCC)

Received: Mar. 24, 2020
Revised: May 13, 2020
Editor: Zhongyuan Zhao

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

In this paper, we considered a two-layer cellular network with mobility-aware MEs that can improve the efficiency of task processing by means of MEC technology.

allows mobile application tasks to run on remote data center through high speed and reliable air interfaces [6]. Unfortunately, MCC has an inherent drawback, incurring very large latency overhead because of the long-distance propagation through wireless access network and backhaul network, where the routing and management operations in the network may add additional delay overhead [7]. As a result, the MCC architecture is not suitable for modern time-sensitive tasks.

The industry proposed the concept of mobile edge computing (MEC) to solve problems above. MEC deploys computation and storage resource on the network edge to provide computation capacity for the mobile network, establishing low latency and high bandwidth network service solutions [8]. Compared with MCC, MEC avoids the problems caused by long-distance transmission to improve privacy and security of mobile applications, such as the high concentration of information on the platform, vulnerability to attacks, and the disclosure and loss of private data due to the separation of ownership and management rights of user data [9].

The research on computation offloading in MEC network has drawn wide attention from academia at home and abroad. MEC gives mobile developers and content providers the ability to sink cloud computing to the edge by placing servers on the base station side [10]. Due to the base station network resources and computation resources are limited, [11] proposed a bandwidth allocation scheme based on collectable information to meet the requirements of each flow in the case that offloading tasks need to compete with each other. Aiming at the problem of how to determine the offloading tasks reasonably and assign the resources properly, it was presented in [12] a scheme of joint optimization of offloading strategy, wireless and computational resource allocation, which can effectively improve the profits of network operators by solving two sub-problems iteratively and obtaining sub-optimal solutions. Additionally, technologies such as user virtualization have been introduced into the MEC architecture for collaborative service delivery [13]. In the software-defined virtualization cellular network, [14] proposed a distributed resource allocation algorithm based on alternating direction multiplier by virtualizing wireless, computing and storage resources together to reduce the computational complexity and signaling overhead.

However, few articles have considered the offloading optimization problem with mobility-aware mobile equipment (MEs) in mind. Because of the movement, MEs have the potential to get out of the small cell network (SCNs). If they offload tasks onto the edge nodes, the calculated results need migration to the next small-cell base station (SBS) capable of serving MEs [15]. As mentioned in [16], MEC servers can be shared to reduce switching and migration time. [17] divided metropolitan area into disconnected MEC areas based on user movement to minimize the possible migration. In [18], it considered a super-dense network with many SBSs. Due to the mobility of users, it assigned tasks to many SBSs in different locations and proposed a user-centered movement management scheme based on energy perception.

At the same time, reinforcement learning (RL) was introduced into the MEC network to solve the low efficiency of traditional optimization methods. Since RL maximizes long-term returns, it supports the MEC network for dynamic task arrival. [19] proposed a reinforcement learning-based framework to work out the resource allocation problem of MEC system, minimizing the consumption of energy and delay by optimizing the allocation of computation resource and network resource jointly. A RL-based resource allocation scheme was considered in [20], which allows the cloud to decide whether to accept requests and allocate resources. [21] and [22] respectively used deep reinforcement learning (DRL) to allocate wireless resources and make maintenance decisions for Internet of things (IoT) devices and meet various QoS requirements of different applications, proving the superiority of DRL scheduling in multi-service scenarios.

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

In this paper, we introduce the background and research status of MEC in detail, and then model the system based on the characteristics of user's mobility in MEC network. A joint problem of task offloading, migration and resource allocation is presented and then solved by classical Q-learning algorithm of reinforcement learning. Finally, the simulation results are compared with other traditional baselines to confirm its significant improvement on total revenue of MEs. The main contributions include following aspects:

- A mobility model of MEs based on sojourn time in each SCNs is considered, which follows the exponential distribution. MEs leaving the cell without completing offloaded tasks will cause the execution results to be migrated. On basis of the mobility model, joint task offloading and migration schemes in mobility-aware MEC network are proposed, which can lower the migration frequency and raise the system revenue.
- The joint optimization problem is considered as a mixed integer non-linear programming (MINLP) problem and it is NP-hard, too. Then we put forward a RL-based theme for solution. As the basis of reinforcement learning, we respectively define the three elements of Markov Decision Process (MDP), namely state, action and reward. Furthermore, the classical Q-learning algorithm is used to solve the problem.
- Compared with other schemes, we verified the performance of proposed Q-learning method, demonstrating the effectiveness of the proposed reinforcement learning-based theme. The theme is capable of achieving excellent revenue-raising performance gain in the case of different system characteristics.

The paper is organized as follows. Firstly, we present the mobility-aware MEC system model, including mobility model in Section II. Then the utility function of the optimization problem is derived according to two typical cases in Section III. In Section IV, the basic theory of reinforcement learning and the algorithm applied in this paper are introduced in detail. In Section V, we give the simulation results and analysis compared to other baseline schemes. Finally, the above contents are summarized in Section VI.

II. System Model

In this section, a system model of mobility-aware MEC network is proposed by describing the network model, computation model and migration model successively.

2.1 Network model

As shown in Figure 1, we consider a two-layer cellular network with a macro base station (MBS) and many SCNs, each of which equipped with a SBS. The MEC server is deployed at the center of SCN attached to SBS, and MEs are randomly distributed in the service area. As MEs move, they are not stationary and may leave the current service cell. Moreover, due to the particularity of different MEs, their states in the network are also very different. The time of some MEs on task computation is less than the time overridden by the current SCN, such as ME_1. While other MEs move to another cell when the tasks have not finished processing, such as ME_2. For ME_2, the computation results have to be migrated between SBS, causing extra cost for migration.

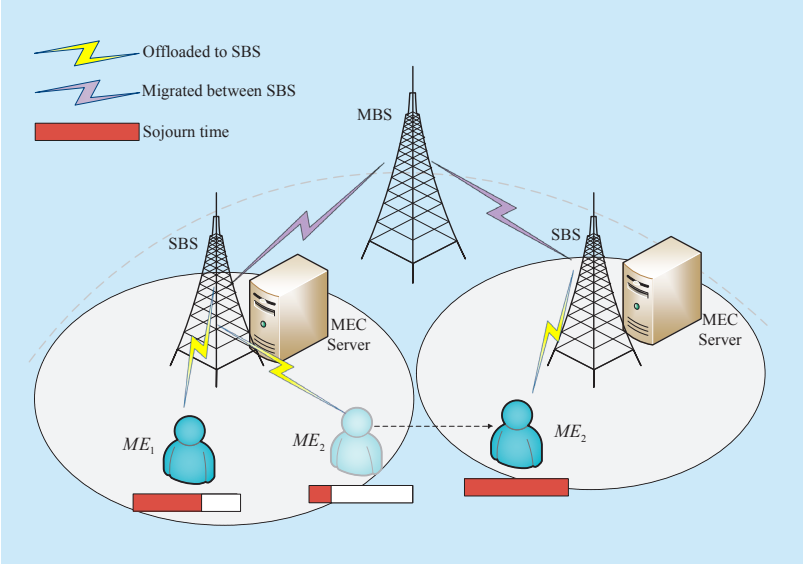

Fig. 1. *System model of proposed mobility-aware MEC network.*

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

We denote $\mathcal{ME}=\{ME_1, ME_2, ..., ME_N\}$ as the set of MEs in the cellular network and number it as $\mathcal{N}=\{1,2,...,N\}$. The task of ME_i can be described as $S_i=\{H_i, D_i, T_i^{max}\}, \forall i \in \mathcal{N}$, where H_i is the size of the computation data, D_i denotes the asked computation resource and T_i^{max} stands for the maximum latency of the task. In particular, D_i is measured by CPU revolutions and satisfies the condition that $D_i = \varepsilon H_i$ [23]. To simplify the calculation, the transmission power of ME_i is assumed as a constant, denoted by p_i. And d_i, h_i present the distance from ME_i to SBS and the channel gain, respectively. Therefore the signal-to-noise ratio (SNR) is given by

$$SNR_i = \frac{p_i h_i d_i^{-\theta}}{\sigma^2}, \tag{1}$$

where θ is the standard path loss propagation exponent and σ^2 denotes the power of additive white Gaussian noise. So we have the uploading transmission rate of ME_i

$$R_i = B\log_2(1 + SNR_i), \tag{2}$$

where B is the bandwidth for communications between MEs and SBS.

2.2 Computation model

Due to the computation-intensive tasks requested by MEs, offloading these kinds of tasks can not only speed up the process, but also save time and energy consumption for devices. In this paper, tasks can be locally calculated by MEs or handled on the MEC server at SBS side. Denote $a_i \in \{0,1\}(\forall i \in \mathcal{N})$ as the task offloading decision of ME_i, where $a_i = 1$ as the task offloaded to MEC server and $a_i = 0$ as the task executed locally. Thus, $A=\{a_i\}_{i\in\mathcal{N}}$ is the set of offloading decisions of MEs.

1) Local Computation: If $a_i = 0$, MEs calculate the computation tasks locally by their distinct computational capabilities, set as f_i^L. So the computation time t_i^L executed by ME_i is expressed as

$$t_i^L = \frac{D_i}{f_i^L}. \tag{3}$$

And the local energy consumption for computation [24] can be given as

$$E_i^L = \kappa D_i \left(f_i^L\right)^2, \tag{4}$$

where κ is the factor about switched capacitance [25].

2) Edge Computation: If $a_i = 1$, ME_i offloads the computation task to the edge node through wireless channel between MEs and SBS. And the task is computed on the MEC server connected to the SBS. The process of offloading task to the MEC server can be viewed as transmission and execution. We denote the transmission time for uploading data to MEC server as $t_i^{mt} = H_i / R_i$. Compared with the data that needs to be uploaded, the size of proceeding result is so tiny that the transmission time to download or migrate can be ignored [26]. From MEs' point of view, there is no need to consider the power consumption of task calculation and result transmission, so the energy consumption E_i^M is

$$E_i^M = E_i^{mt} = p_i t_i^{mt}. \tag{5}$$

Denote $F=\{f_i\}_{i\in\mathcal{N}}$ as the set of MEC computation allocation and f_i is the edge computation resource assigned to ME_i. So the execution time is $t_i^{me} = D_i / f_i$. Because the capacity of MEC server C_{MEC} is not unlimited, so some of the tasks cannot be offloaded, and the resource allocation must satisfy the constraint $\sum_{i=1}^{N} f_i \le C_{MEC}$. Therefore, the computation time in this case is

$$t_i^M = t_i^{mt} + t_i^{me}. \tag{6}$$

2.3 Mobility model

Since SBS is located in the center of small cell, once MEs leave the area where service is currently being provided, i.e. d_i exceeds the radius of SCNs R_s, resulting in the migration of tasks due to cell switching. If the above situation never occurs, that is, when $d_i \le R_s$ is always true, it means that MEs will not leave the region until tasks are completed. Therefore, we use a period of time to record the duration before d_i breaks the limit, which can be counted as the cell sojourn time of MEs.

Cell sojourn time refers to the time spent by

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

mobile users in a given cell and is a significant performance measure for planning network resources and improving QoS [27]. Here, we use an exponential function of sojourn time to measure the mobility of MEs [28]. So the probability density function of sojourn time $p_{\tau_i}(t)$ can be given as

$$p_{\tau_i}(t) = \frac{1}{\tau_i} e^{-\frac{t}{\tau_i}}, t \geq 0, \tag{7}$$

where τ_i means the average sojourn time of ME_i and it varies among different MEs. We regard τ_i follows the Gaussian distribution for simplicity, and the reliable τ_i can be obtained by collecting historical data of MEs.

Note that τ_i is different from another concept T_i^{max}. We introduce it to measure the mobility characteristics of MEs, according to which value, the trajectory of MEs in different situations can be determined. When τ_i relatively small, it may cause task migration between SBSs, increasing the complexity of the task offloading. Obviously, the premise of all this is that the task processing time of MEs does not exceed T_i^{max}. Once MEs cannot receive the result before the maximum delay of the task, it is not only a failure for MEs, but also a waste of resources to the whole system.

III. Problem Statement

In this section, we give a utility function that describes the problem of offloading resource optimization, including the revenue and the cost. The revenue consists of two parts: spare time and spare local energy. Spare time refers to time saved due to edge computation, which is

$$G_i^T = \theta_t \left(t_i^L - t_i^M\right), \tag{8}$$

where θ_t is the coefficient of spare time revenue. In the same way, we describe spare local energy as

$$G_i^E = \theta_E E_i^L, \tag{9}$$

where θ_E is the coefficient of revenue by saving local energy and can be described as the price per unit of energy consumption, too.

Meanwhile, sources of cost include MEs selection of edge computation mode and possible task migration. The resource cost of selecting the offloading mode include the energy consumption of the data to be transferred and the MEC execution resource to be allocated. So we have total resource cost

$$Z_i^M = \theta_E E_i^M + \theta_f f_i, \tag{10}$$

where θ_f is the price per unit of MEC execution resource to be allocated.

According to the time-length relationship between the edge-calculated time and the estimated sojourn time, the utility function of edge computation mode is discussed in the following two cases.

1) Before leaving the current cell, the tasks of MEs are smoothly finished on the MEC server. This case can be described as the whole process of task offloading time is less than the sojourn time of MEs, e.g. ME_1 in Figure 1. That is, no migration of tasks occurs. According to (7), the probability of this situation can be described as $P_{\tau_i}(t > t_i^M)$. Therefore, the utility function of the corresponding part is

$$Q_i^1 = G_i^T + G_i^E - Z_i^M. \tag{11}$$

2) Because of mobility-aware MEs, there is a situation on the MEC server where MEs have a shorter stay in the previous cell, e.g. ME_2 in Figure 1. So they leave the cell before tasks are completed, with the probability of $P_{\tau_i}\left(t \leq t_i^M\right)$, and the execution results cannot be directly passed back to MEs. Instead, the results must be transmitted to another SBS in the new SCN where MEs reside, and then migrated back. This process of transmission between SBS will incur additional migration cost Z_i^m and it's recognized to be related to the size of the computation data H_i, given as $Z_i^m = \delta H_i$ [16]. So the utility function of the corresponding part is denoted as

$$Q_i^2 = G_i^T + G_i^E - Z_i^M - Z_i^m. \tag{12}$$

Combine (11) and (12), we can get the expression of the utility function as

$$Q_i = \begin{cases} Q_i^1, & t > t_i^M \\ Q_i^2, & t \leq t_i^M \end{cases}, \forall i \in \mathcal{N}. \tag{13}$$

According to the mobility model, the corresponding probability is calculated as

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

$$P_{\tau_i}(t \le t_i^M) = \int_0^{t_i^M} \frac{1}{\tau_i} e^{-\frac{t}{\tau_i}} dt = -e^{-\frac{t_i^M}{\tau_i}} + 1, \quad (14)$$

$$P_{\tau_i}(t > t_i^M) = 1 - P_{\tau_i}(t \le t_i^M). \quad (15)$$

Therefore, we propose the expectation of utility $\overline{Q}_i$ to describe the performance of ME_i

$$\overline{Q}_i = P_{\tau_i}(t > t_i^M) Q_i^1 + P_{\tau_i}(t \le t_i^M) Q_i^2, \forall i \in \mathcal{N}. (16)$$

Specially, we set $\overline{Q}_i = 0$ when $a_i = 0$, because MEs generate no revenue from MEC server when tasks are computed locally. As a result, we have the joint optimization problem of tasks offloading decisions and computation resource allocation, which can be described as

$$\begin{aligned} &\max_{A,F} \sum_{i \in \mathcal{N}} a_i \overline{Q}_i(f_i) \\ &s.t. C1: a_i \in \{0,1\}, \forall i \in \mathcal{N} \\ &\quad C2: 0 < t_i^M \le T_i^{\max}, \forall i \in \mathcal{N}. \\ &\quad C3: f_i \ge 0, \forall i \in \mathcal{N} \\ &\quad C4: \sum_{i \in \mathcal{N}} f_i \le C_{MEC} \end{aligned} \quad (17)$$

Constraint (C1) ensures that each MEs selects local computation or edge computation. In (C2), it makes sure that the time of edge computation must be positive and it doesn't exceed the deadline of the task delay. (C3) ensures that MEC computation resource distributed to each MEs is nonnegative. While constraint (C4) guarantees that the allocated computation resource doesn't exceed total quantity of MEC capacity.

Because of the integer constraint $a_i \in \{0,1\}$ of (C1), problem (17) is a MINLP problem. The feasible set and the objective function of it are non-convex, so it is also NP-hard. In order to solve problem (17), we propose the reinforcement learning method to find optimal A and F, instead of operating the traditional optimization methods for NP-hard problems.

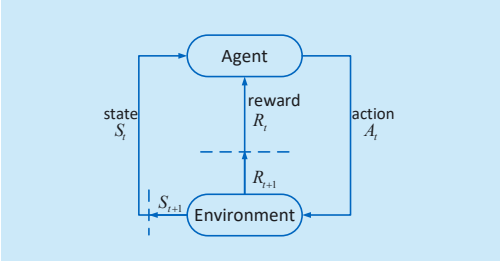

Fig. 2. *Five-tuples of Markov Decision Process to describe reinforcement learning: <"Agent", "Environment", state, action, reward>.*

IV. Problem Solution

In this section, we first introduce the theory of Markov Decision Process. Then the exact state, action and reward of reinforcement learning solution in this paper are defined respectively. Furthermore, the optimization algorithm based on classical reinforcement learning algorithm Q-learning is introduced in detail.

4.1 Markov decision process

In mobile edge computing network, traditional solutions to joint distribution problem for tasks offloading and resource allocation include exhaustive search algorithm and game theory [29]. However, these methods also exist many limitations. On one hand, high algorithm complexity leads to low efficiency. On the other hand, heavy computational resource consumption and low fault tolerance make it difficult to apply to largescale network scenarios. In this paper, reinforcement learning method is used for solving joint optimization problem to avoid limitations and abuses of traditional algorithms to some extent.

The general model of reinforcement learning is on a basis of Markov Decision Process, which aims to provide an intuitive framework for learning from interactions to achieve goals. In reinforcement learning, the "Agent" refers to the learner or decision maker. Anything else that has interactional effect with the "Agent" other than itself is called "Environment". During this interaction, the "Agent" selects and performs one action in a state by policy, and the "Environment" responds to this action by moving the "Agent" into the next new state, then it gives feedback on the action and generates a reward.

After continuous iterations, the "Agent" optimizes the reward value by selecting different actions to converge to an optimal state. As shown in Figure 2, reinforcement learning is carried out in the dynamic interaction between the "Agent" with the "Environment".

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

We denote $\mathcal{S}$ as state space, $\mathcal{A}$ as action space and $\mathcal{R}$ as reward space. Without special instructions, the problem solved by reinforcement learning is generally based on a finite MDP model. That is, MDP's state, action and reward space are all finite set [30]. The decision of the "Agent" on specific tasks, such as the action to choose under the state, is determined and executed according to a policy function. Generally speaking, the policy function is mostly a kind of probability distribution [31]. In each interaction, the "Agent" has an initial state s_t, in which it selects and executes the action a_t according to the policy function. The "Environment" gives feedback to the action, then generates a reward value r_{t+1} and causes the "Agent" to enter the next new state s_{t+1}, which will be used by the "Agent" as the new initial state for the next round of interaction. The next state of the "Agent" is random, and its state transition probability has Markov property. Markov property can be understood as a property of state transition probability independent of previous history. The "Agent" makes new decisions based on the newly observed states, which are repeated in turn, and then iterates for several rounds until the final convergence.

4.2 three key elements for RL

In the system model of this paper, we assume that MEs act as the MDP decision-maker (i.e., the "Agent" described above), attempting to access the MEC server nearby to interact with the "Environment". The "Environment" consists of channel conditions and MEC servers. Throughout the interaction, MEs seek to maximize the system's utility function despite the uncertainty of the "Environment". The action of MEs can affect future state of the "Environment", thus further impact the action choice and state space of the next time step of MEs. In the case of results partly random and controlled by the decision-maker, MDP provide a good mathematical framework for modeling [32], and then complete the whole decision-making process by means of reinforcement learning.

More specifically, the process of task assignment decision is modeled as a MDP. Each interaction between the "Agent" and the "Environment" is called an episode and it is equivalent to the offload period T. An episode can be divided into multiple time steps and the terminal device has a state s_t at each time step $t\left(t=1,2,...\right)$. In each episode, MDP iterates from a random initial state until it finally converges. MEs as the decision maker select

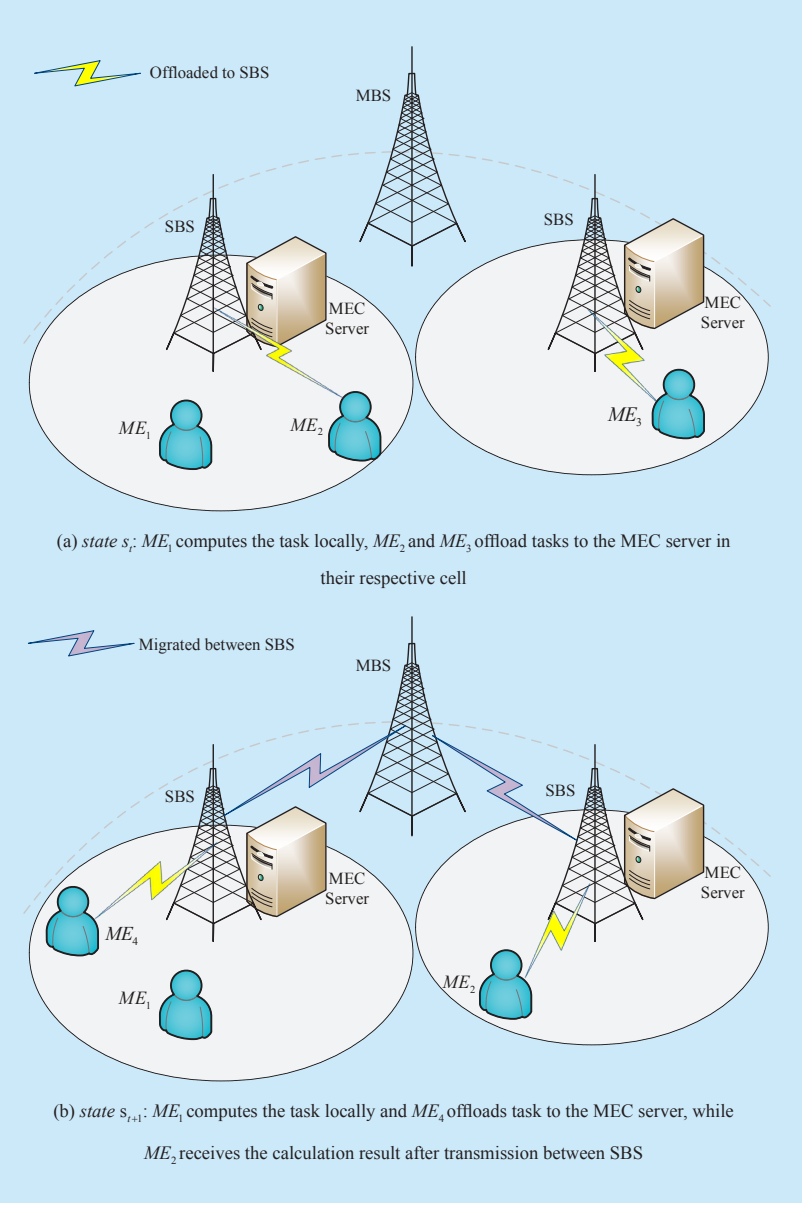

Fig. 3. *Possible state of the mobility-aware MEC network at different time step, which is related to the number of MEs in the SCNs, their movement status and tasks property.*

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

an action from the optional action under state s_t according to different strategies adopted by different algorithms and executes it. Meanwhile, the MEC server as part of the environment responds and gives a corresponding reward to MEs, and then MEs enter the next state s_{t+1}. As mentioned above, the future state in this MDP only depends on the current state and has nothing to do with the historical state, which guarantees the memory-free Markov property.

The following is the exact definition of the state, action and reward function of RL used in the joint allocation algorithm:

1) State: For purpose of the goal to maximize total revenue of the entire system, the state for the network scenario needs to reflect the number of unsolved tasks requested by MEs in the current system and the status of mobility-aware MEs in SCNs. Therefore, state mainly consists of three quantities: the number of MEs, the property of the unsolved tasks and the movement of MEs. The number of MEs S^n refers to the quantity of MEs in different cell; the property of the unsolved tasks $S^d = \left[S_1^d, S_2^d, \ldots, S_N^d\right]$ refers to the size of the required computation resource of task; the mobility of MEs $S^m = \left[S_1^m, S_2^m, \ldots, S_N^m\right]$ is used to represent the user movement state. For our wireless network scenario, state can finally be defined as $\mathcal{S} = \left[S^n, S^d, S^m\right]$.

2) Action: For each time step t, MEs must select and execute an action in the current state according to the policy used. We operate the greedy algorithm as the policy here, which is common and easy to implement. Meanwhile, MEs move from the present state s_t to the next state $s_{t+1}\left(\forall s_t, s_{t+1} \in \mathcal{S}\right)$. We denote $\mathcal{A} = \left[A_1, A_2, \ldots, A_N\right]$ to represent the action space in MDP. Correspondingly, for $ME_i\left(i \in \mathcal{N}\right)$, $A_i = 0$ represents that the computation task is selected to perform locally, and $A_i = 1$ implies that the task is calculated by the assigned MEC server in the current cell.

3) Reward: After each "Agent-Environment" interaction, MEs as the "Agent" will get feedback from the "Environment", which is the reward r. It is used to reflect the good or bad results of an action performed by MEs in a certain state. Generally, the reward space of RL should be related to the optimization formula. At the same time, the goal of our optimization problem is to find the maximum benefit of all MEs, while the objective of RL is to get the maximum return. So due to the positive correlation between them, we formulate the function of reward as $\mathcal{R} = \varpi \sum_{i \in \mathcal{N}} a_i \bar{\mathcal{Q}}_i\left(f_i\right)$, where ϖ is the coefficient of system revenue.

4.3 Q-learning based algorithm

Q-learning is a classical value-based RL algorithm. $Q(s,a)$ (i.e. Q value) is the expectation benefit that an action a can be taken to obtain of state s. For each step, the environment provides the agent a reward r according to its action. Thus, the key step of Q-learning algorithm is establishing a Q-table, which corresponds to the Q value obtained from state-action pairs. The most profitable choice is then incurred on a basis of the Q-table and the agent is able to obtain the maximum benefit according to the Q value. $Q(s,a)$ can be given as:

$$\begin{aligned} &NewQ(s,a) = Q(s,a) \\ &+\alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]' \end{aligned} \tag{18}$$

where s, a is the present state and action, s', a' is the next state and action. And α, γ are parameters of Q-learning that satisfy $0 \le \alpha, \gamma \le 1$. α is introduced to measure the value of the last learning versus the current learning. Set it too low and the agent focuses only on the previous knowledge and not on the new reward. Usually, 0.5 is used to balance prior knowledge with new reward. And γ is defined as the learning parameter. It is worth noting that $\gamma \to 0$ indicates the agent is more likely to consider the immediate reward, and $\gamma \to 1$ represents that the agent is conversely focused on the future reward. Generally, we take $\gamma = 0.9$, which can fully consider external rewards. Depend on the description of Algorithm 1, it is shown the procedure of RL-Based Algorithm with Q-learning method.

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

In order to obtain correct convergence, Q-learning actually requires all state-action pairs $Q(s,a)$ to update continuously and $Q(s,a)$ has been verified to be roughly consistent with optimal Q value with infinite strategy for exploration [33]. As a result, Q-learning will find the optimal strategy of action choice for each step after infinite times of exploration. If $\mathcal{I}$ is used to represent the number of iterations of Algorithm 1, then the computational complexity of the algorithm can be expressed by $O(\mathcal{I}\mathcal{N}^2)$. In this paper, we operate the ε−greedy policy to randomly explore the action in the state space closed to finite, so as to get a nearly optimal scheme.

Algorithm 1. RL-Based Algorithm with Q-learning method.

Set the initial $Q(s,a)$
  for each episode T: **do**
    Assign state s_t randomly
    for each time step t: **do**
      Select an action a_t for present state s_t Execute a_t and compute Q value $Q(s_t,a_t)$
      Renew Q-table:
      $Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha\left[R_{t+1} + \gamma\, max_a\, Q(s_{t+1},a_t) - Q(s_t,a_t)\right]$
      Let $s_t \leftarrow s_{t+1}$
      until reach expected state $s_{terminal}$
    end for
  end for

V. Simulation Result

To illustrate the performance of proposed RL-Based Algorithm (RLBA), we consider $R_S = 80m$ in mobility-aware MEC network [29]. Assume the transmission power of MEs follows $N(\mu_1, \sigma_1^2)$, where $\mu_1 = 20dBm$ and $\sigma_1 = 2$. Considering the mobility model, the average sojourn time of MEs follows $N(\mu_2, \sigma_2^2)$, where $\mu_2 = 40seconds$ and $\sigma_2 = 20$. According to [19], we set the parameters of reinforcement learning, where $\alpha = 0.5$ and $\gamma = 0.9$. In addition, other simulation parameters related to the system model are listed in TABLE I [23] [34].

Table I. *Simulation parameters.*

Parameter	Value
R_s	$80m$
B	$10MHz$
μ_1	$20dBm$
σ_1	2
θ	2
H_i	$[100,150]MB$
ε	$[0.4,0.5]Gcycles/MB$
T_i^{max}	$80s$
θ_E	$8units/Joule$
θ_f	$2units/GHz$
θ_t	$4units/second$
μ_2	$40seconds$
σ_2	20
κ	10^{-11}
α	0.5
γ	0.9

The performance of RLBA is compared to other three different algorithms. The Genetic Algorithm-Based Algorithm (GABA) is a traditional suboptimal algorithm to solve the optimization problem due to its good performance on global search. Both Randomly Offloading Algorithm (ROA) and Fully Offloading Algorithm (FOA) take the mobility of MEs into account. ROA stands for MEs randomly select local computation or edge computation and FOA means that all tasks are offloaded to the edge node. Moreover, the ratio of the offloaded tasks varies, the former has a probability of 0.5, while the latter has a probability of 1.

The convergence performance of RLBA and GABA with different amounts of MEs is shown in Figure 4. It's shown that both in the proposed Q-learning-based method and GABA method, the total revenue of MEs increases with the amounts of iterations until it reaches a relatively stable value, which means the algorithms are convergent. In the case of a small number of MEs, the convergence speed of Genetic Algorithm is fast, usually within 1,000 iterations. But we can observe that once RLBA converges, it always gains higher system revenue than GABA. Moreover, if the

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

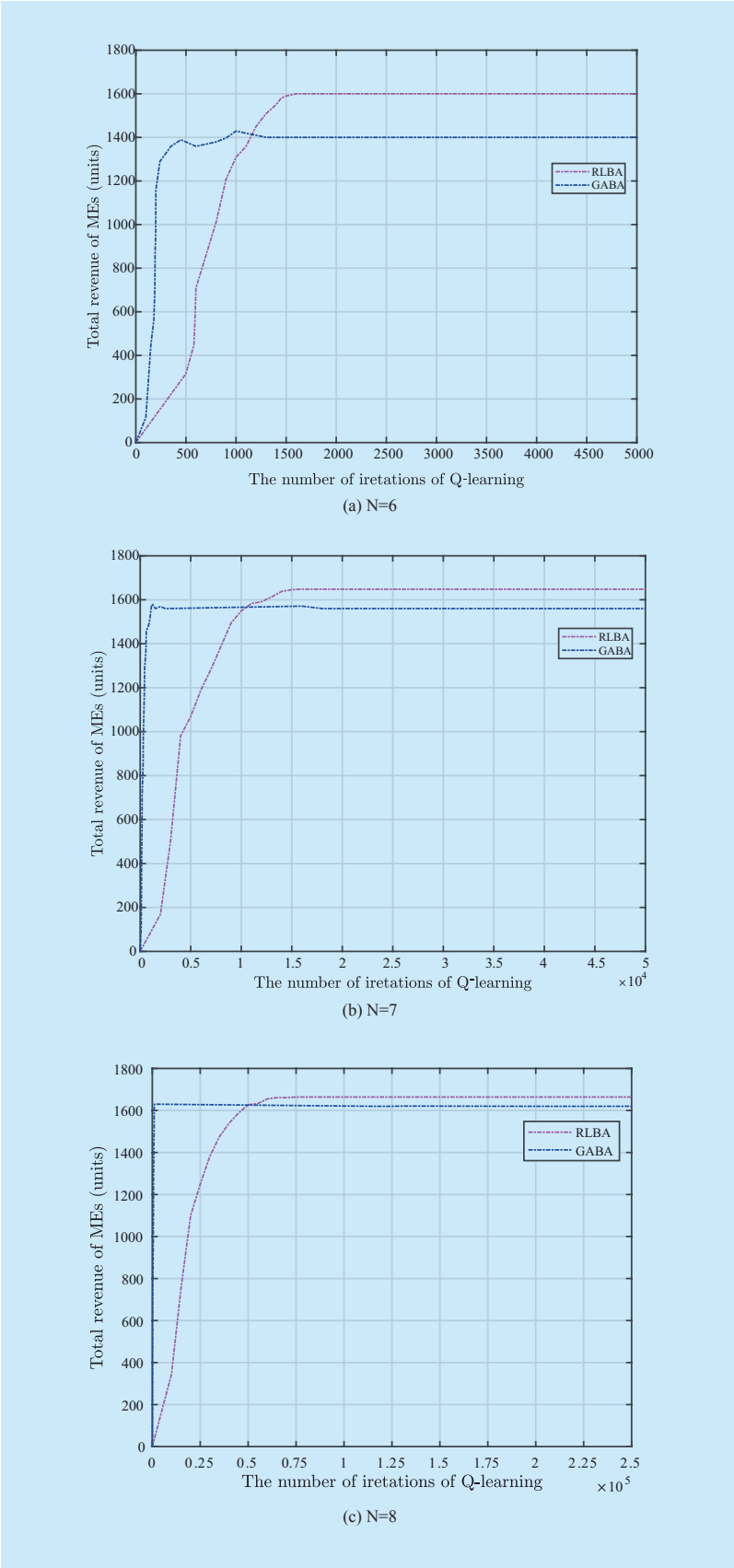

Fig. 4. *The convergence performance of the proposed RLBA scheme and GABA with different amounts of MEs ($C_{MEC} = 18GHz$ and $\delta = 0.5units / MB$).*

maximum capacity C_{MEC} and the cost of migration δ are all constant, the number of MEs in the coverage range of SCNs has a great impact on the convergence property of RLBA. This is reflected in the rapid growth trend in the quantity of iterations. When the number of MEs reaches 6, it takes only about 1,500 iterations for the function to converge. On condition that the number of MEs grows by one, the minimum number of iterations increases to about 15,000, even explodes to approximately 75,000 if one more is added. Compared to the original case, that's 10 times and 50 times the number of iterations, respectively. Thus, it can be predicted that the number of iterations to converge will gradually increase as the quantity of MEs raises.

Figure 5 presents the total revenue of MEs under gradually rising quantities and the performance varies greatly. As the number of MEs increases from 4 to 8, RLBA, GABA and ROA maintain a continuous growth trend, while FOA increases at first and then decreases soon. For RLBA and GABA, computation resource is first plentiful so that more tasks can be offloaded as the amount of MEs increases. However, when the number reaches 5, the total revenue begins to change slowly. Because the computation capacity of MEC is not unlimited, it only provide service to a certain number of MEs, and more MEs leads to more tasks and more decision options. So the total revenue goes up at a slower pace. The reason for ROA is similar to them, but the total revenue is lower due to the small quantity of offloaded tasks. In terms of FOA, system revenue increases first due to the abundance of resources. But as migration tasks become more frequent, the incremental cost of migration will be greater than the incremental revenue of saving time and energy. When there are six MEs in the system, total revenue starts to decline at the similar speed. However, as the quantity of users increases, the migration cost of FOA is still increasing. The revenue of spare time starts to decline, so the total revenue begins to decrease.

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

The total revenue of different edge computation capacity is shown in Figure 6. It's represented that with the improvement of the MEC computation resource, each algorithm gains a higher revenue. However, their growth rates are different, depending on the amounts of offloaded and migrated tasks. Based on feedback from the environment, RLBA and GABA selectively offload tasks through different strategies to reduce migration cost. And because of that, they can get higher total revenue than others. When the capacity of the MEC server is sufficient, both algorithms will try to improve the revenue by edge offloading as much as possible. Compared with GABA, the higher total revenue of RLBA is achieved by better offloading decisions through the Q-learning algorithm. Furthermore, total revenue of ROA grows slowly. ROA's growth rate is close to that of the two optimal algorithms, but its revenue is slightly lower due to the uncertainty of offloading tasks. The reason for the increasing speed of FOA is different. The offloaded tasks of FOA will not change, but with the improvement of MEC computing capacity, more and more tasks don't need to be migrated any more, and the reduction of migration cost can quickly improve the total revenue.

In Figure 7, total revenue all decline as migration cost increases. Among them, RLBA and GABA decrease slowly, that is, the increase of migration has no obvious effect on migration probability, which indicates that these two schemes can reduce the migration probability. It can be observed that the total revenue of RLBA and GABA is higher than other three schemes, and RLBA performs slightly better than GABA. Once again, the better performance of reinforcement learning over traditional methods is verified. In the case of ROA, there are not many MEs that select the edge node to calculate tasks, so the migration probability is relatively low. In terms of FOA, most of MEs choose edge computation mode, resulting in a shortage of MEC computation resource and increasing the likelihood of tasks migration. With the increase of migration cost, we can find a significant and rapid decrease in total revenue.

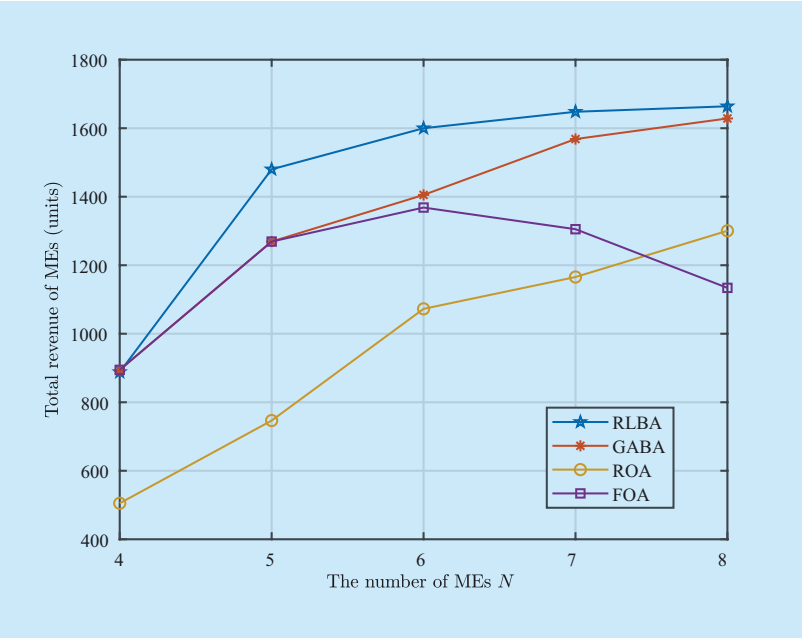

Fig. 5. *The impact of MEs quantities from 4 to 8 ($C_{MEC} = 18GHz$ and $\delta = 0.5units / MB$).*

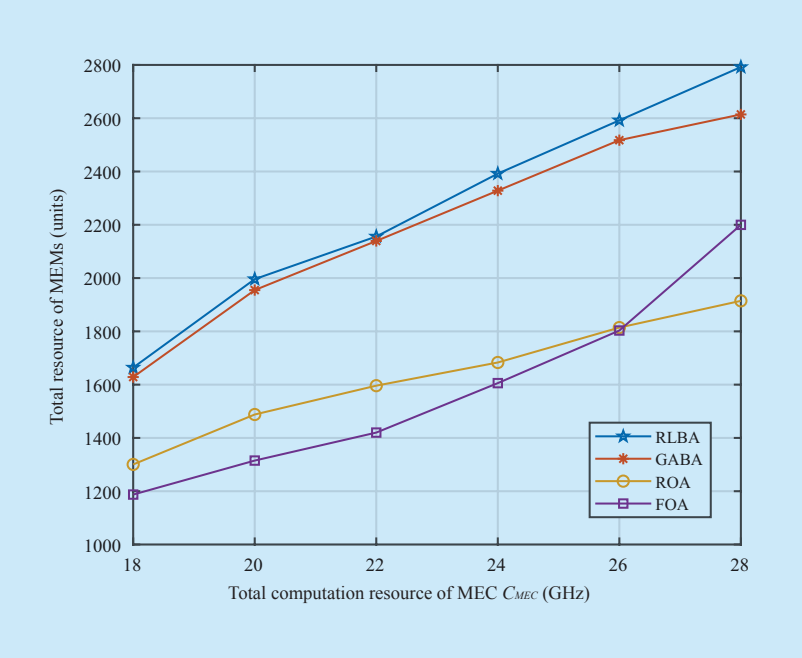

Fig. 6. *The impact of edge computation maximum capacity C_{MEC} from 18 to 28(GHz) ($N = 8$ and $\delta = 0.5units / MB$).*

VI. Conclusion

In this paper, we considered a two-layer cellu-

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

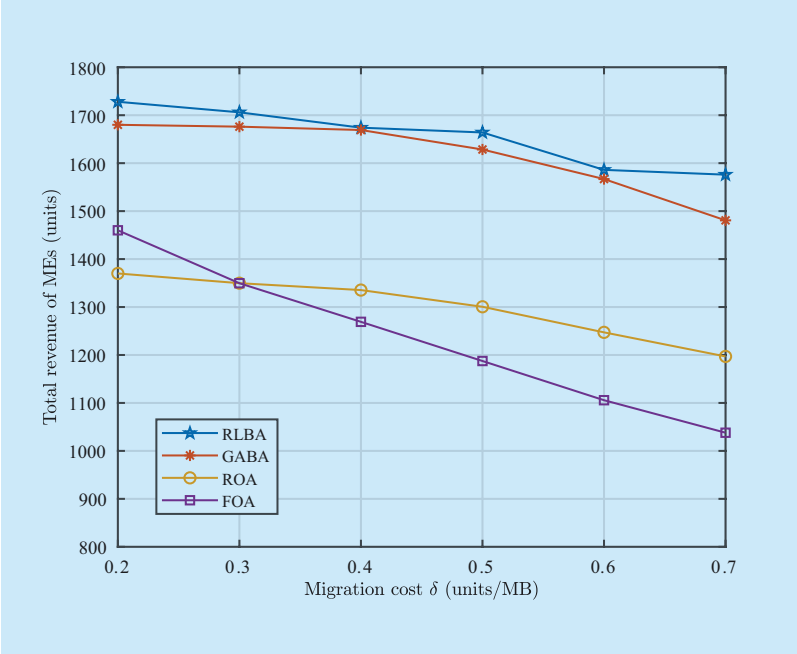

Fig. 7. *The impact of migration cost from 0.2 to 0.7 (units / MB) ($N=8$ and $C_{MEC}=18GHz$).*

lar network with mobility-aware MEs that can improve the efficiency of task processing by means of MEC technology. In order to lower the migration probability and maximize the total revenue, we put forward joint task offloading and computation allocation schemes based on mobility-aware migration. Then it is formulated as a MINLP problem and the RL-Based algorithm we derived to solve this problem can obtain maximum total revenue of MEs availably. Simulation results showed that this algorithm can achieve better performance compared with other baseline algorithms, including traditional methods such as the Genetic Algorithm. Future work is underway to consider active collaborations between multiple SBSs to reduce migration in different ways.

Acknowledgment

This work was supported in part by the National Natural Science Foundation of China under Grant 61701038.

References

[1] Y. Mao, C. You, J. Zhang, et al., "A survey on mobile edge computing: the communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 5, 2017, pp. 2322-2358.

[2] M. Peng, Y. Li, J. Jiang, et al., "Heterogeneous cloud radio access networks: a new perspective for enhancing spectral and energy efficiencies," *IEEE Wireless Communications*, vol. 21, no. 6, 2014, pp. 126-135.

[3] S. Wang, X. Zhang, Y. Zhang, et al., "A survey on mobile edge networks: convergence of computing, caching and communications," *IEEE Access*, vol. 5, 2017, pp. 6757-6779.

[4] Z. Zhao, M. Xu, Y. Li, et al., "A non-orthogonal multiple access-based multicast scheme in wireless content caching networks," *IEEE Journal on Selected Areas in Communications*, vol. 35, no. 12, 2017, pp. 2723-2735.

[5] Z. Zhao, M. Xu, W. Xie, et al., "Coverage performance of NOMA in wireless caching networks," *IEEE Communications Letters*, vol. 22, no. 7, 2018, pp. 1458-1461.

[6] W. Shi, J. Cao, Q. Zhang, et al., "Edge computing: vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, 2016, pp. 637-646.

[7] M. Peng, K. Zhang, J. Jiang, et al., "Energy-efficient resource assignment and power allocation in heterogeneous cloud radio access networks," *IEEE Transactions on Vehicular Technology*, vol. 64, no. 11, 2015, pp. 5275-5287.

[8] N. Abbas, Y. Zhang, A. Taherkordi, et al., "Mobile edge computing: A survey," *IEEE Internet of Things Journal*, vol. 5, no. 1, 2018, pp. 450-465.

[9] J. Cheng, Y. Shi, B. Bai, et al., "Computation offloading in CloudRAN based mobile cloud computing system," Proc. *2016 IEEE International Conference on Communications (ICC)*, 2016, pp. 1-6.

[10] T. Tran, A. Hajisami, P. Pandey, et al., "Collaborative mobile edge computing in 5G networks: New paradigms scenarios and challenges," *IEEE Communications Magazine*, vol. 55, no. 4, 2017, pp. 54-61.

[11] Y. Ito, H. Koga, K. Iida, "A bandwidth allocation scheme to meet flow requirements in mobile edge computing," Proc. *2017 IEEE 6th International Conference on Cloud Networking (CloudNet)*, 2017, pp. 1-5.

[12] J. Zhang, M. Wu, M. Zhao, "Joint computation offloading and resource allocation in C-RAN with MEC based on spectrum efficiency," *IEEE Access*, vol. 7, 2019, pp. 79056-79068.

[13] M. Peng, S. Yan, K. Zhang, et al., "Fog-computing-based radio access networks: issues and challenges," *IEEE Network*, vol. 30, no. 4, 2016, pp. 46-53.

[14] M. Liu, Y. Mao, S. Leng, et al., "Full-duplex aided user virtualization for mobile edge computing in 5G networks," *IEEE Access*, vol. 6, 2017, pp. 2996-3007.

[15] A. Machen, S. Wang, K. Leung, et al., "Live service migration in mobile edge clouds," *IEEE*

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

Wireless Communications, vol. 25, no. 1, 2018, pp. 140-147.
[16] W. Nasrin and J. Xie, "SharedMEC: Sharing clouds to support user mobility in mobile edge computing," Proc. *2018 IEEE International Conference on Communications (ICC)*, 2018, pp. 1-6.
[17] X. Guan, X. Wan, J. Wang, et al., "Mobility aware partition of MEC regions in wireless metropolitan area networks," Proc. *IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2018, pp. 1–2.
[18] Y. Sun, S. Zhou and J. Xu, "EMM: Energy-aware mobility management for mobile edge computing in ultra dense networks," *IEEE Journal on Selected Areas in Communications*, vol. 35, no. 11, 2017, pp. 2637-2646.
[19] J. Li, H. Gao, T. Lv, et al., "Deep reinforcement learning based computation offloading and resource allocation for MEC," Proc. *2018 IEEE Wireless Communications and Networking Conference (WCNC)*, 2018, pp. 1-6.
[20] H. Liu, S. Liu, and K. Zheng, "A reinforcement learning-based resource allocation scheme for cloud robotics," *IEEE Access*, vol. 6, 2018, pp. 17215-17222.
[21] B. Guo, X. Zhang, Y. Wang, et al., "Deep-Q-network-based multimedia multi-service QoS optimization for mobile edge computing systems," *IEEE Access*, vol. 7, 2019, pp. 160961-160972.
[22] J. Ren, H. Wang, T. Hou, et al., "Federated learning-based computation offloading optimization in edge computing-supported internet of things," *IEEE Access*, vol. 7, 2019, pp. 69194-69201.
[23] A. Al-Shuwaili and O. Simeone, "Energy-efficient resource allocation for mobile edge computing-based augmented reality applications," *IEEE Wireless Communications Letters*, vol. 6, no. 3, 2017, pp. 398-401.
[24] S. Guo, B. Xiao, Y. Yang, et al., "Energy-efficient dynamic offloading and resource scheduling in mobile cloud computing," Proc. *IEEE INFOCOM 2016 - The 35th Annual IEEE International Conference on Computer Communications*, 2016, pp. 1-9.
[25] W. Hao and S. Yang, "Small cell cluster-based resource allocation for wireless backhaul in two-tier heterogeneous networks with massive MIMO," *IEEE Transactions on Vehicular Technology*, vol. 67, no. 1, 2018, pp. 509-523.
[26] X. Chen, L. Jiao, W. Li, et al., "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Transactions on Networking*, vol. 24, no. 5, 2016, pp. 2795-2808.
[27] S. Shin, U. Lee, F. Dressler, et al., "Analysis of cell sojourn time in heterogeneous networks with small cells," *IEEE Communications Letters*, vol. 20, no. 4, 2016, pp. 788-791.
[28] X. Liu, J. Zhang, X. Zhang, et al., "Mobility-aware coded probabilistic caching scheme for MEC-enabled small cell networks," *IEEE Access*, vol. 5, 2017, pp. 17824-17833.
[29] J. Zhang, W. Xia, F. Yan et al., "Joint computation offloading and resource allocation optimization in heterogeneous networks with mobile edge computing," *IEEE Access*, vol. 6, 2018, pp. 19324-19337.
[30] D. Djonin and V. Krishnamurthy, "MIMO transmission control in fading channels - a constrained Markov decision process formulation with monotone randomized policies," *IEEE Transactions on Signal Processing*, vol. 55, no. 10, 2007, pp. 5069-5083.
[31] M. Terefe, H. Lee, N. Heo, et al., "Energy-efficient multisite offloading policy using Markov decision process for mobile cloud computing," *Pervasive and Mobile Computing*, vol. 27, 2016, pp. 75-89.
[32] R. Sutton, A. Barto, "Reinforcement learning: an introduction," 1998.
[33] X. Liu, Z. Qin, Y. Gao, "Resource allocation for edge computing in IoT networks via reinforcement learning," Proc. *2019 IEEE International Conference on Communications (ICC)*, 2019, pp. 1-6.
[34] Z. Tan, F. Yu, X. Li, et al., "Virtual resource allocation for heterogeneous services in full duplex-enabled SCNs with mobile edge computing and caching," *IEEE Transactions on Vehicular Technology*, vol. 67, no.2, 2018, pp. 1794-1808.

Biographies

Dongyu Wang, received the B.S. degree in 2008 and M.S. degrees in 2011, respectively, both from Tianjin Polytechnic University, China, and the Ph.D. degree in 2014 from Beijing University of Posts & Telecommunications, China. From September 2014 to June 2016, he was a Post Ph.D with Department of Biomedical Engineering, Chinese PLA General Hospital, Beijing. In September 2016, he joined the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications. His research interests include device-to-device communication, multimedia broadcast/multicast service systems, resource allocation, theory and signal processing, with specific interests in cooperative communications, Fog computing, and mobile edge computing.

Xinqiao Tian, received the B.S. degree in communications engineering from Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2018. She is currently pursuing the M.E. degree at Beijing

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.

University of Posts and Telecommunications (BUPT), Beijing, China. Her current research interest is about mobile edge computing in wireless communication networks.

Haoran Cui, received the B.S. degree from Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2019. He is currently pursuing the M.E. degree with Beijing University of Posts and Telecommunications (BUPT), Beijing, China. His current research interests include deep reinforcement learning, Machine Learning, mobile edge computing systems.

Zhaolin Liu, received the B.S. degree in communications engineering from University of Electronic Science and Technology of China (UESTC) in 2017. Now he is pursuing his M.E. degree at Beijing University of Posts and Telecommunications (BUPT). His interest is about Edge computing including Mobile Edge Computing and Fog Computing.

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 02:45:41 UTC from IEEE Xplore. Restrictions apply.